

LAPORAN PRATIKUM
ALGORITMA DAN PEMROGRAMAN

PEKAN 9

Disusun Oleh:

Khairun Nisa

2511532015

Dosen Pengampu:

DR. WAHYUDI, S.T, M.T

Asisten Pratikum:

Sherly Sukmadira



FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan kepada Tuhan Yang Maha Esa atas terselesaikannya laporan praktikum Algoritma dan Pemrograman ini. Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum yang telah dilaksanakan pada tanggal 8 Juni 2026. Melalui laporan ini, penulis berharap dapat memberikan gambaran yang jelas mengenai PEKAN 9.

Penulis mengucapkan terima kasih kepada Bapak Dosen serta asisten laboratorium yang telah membimbing selama praktikum berlangsung. Penulis menyadari sepenuhnya bahwa laporan ini masih jauh dari kata sempurna, baik dari segi penyusunan, bahasa, maupun kedalaman materi. Oleh karena itu, penulis dengan rendah hati menerima segala kritik dan saran yang membangun demi perbaikan dan penyempurnaan laporan di masa mendatang. Semoga laporan ini dapat bermanfaat bagi pembaca dan menjadi referensi untuk perkembangan ilmu pengetahuan di teknologi informasi.

Padang, 9 Juni 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan Pratikum	1
1.3 Manfaat Pratikum	2
BAB II PEMBAHASAN	3
2.1 Node_2511532015	3
2.2 BTree_2511532015	5
2.3 BTreeDriver_2511532015	8
2.4 GraphTraversal_2511532015	11
BAB III KESIMPULAN	20
3.1 Kesimpulan	20
DAFTAR PUSTAKA	21

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam mempelajari struktur data, kita tidak hanya mengenal struktur data linier seperti array, linked list, stack, dan queue, tetapi juga struktur data non-linier yang banyak digunakan untuk merepresentasikan data yang memiliki hubungan hierarkis maupun jaringan. Dua struktur data non-linier yang sangat penting dan sering dijumpai dalam dunia pemrograman adalah Tree (pohon) dan Graph (graf). Pemahaman terhadap kedua struktur ini menjadi dasar yang kuat sebelum melangkah ke materi yang lebih kompleks seperti database, routing network, hingga artificial intelligence.

Binary Tree merupakan salah satu bentuk tree yang paling dasar, di mana setiap simpul (node) hanya boleh memiliki paling banyak dua anak, yaitu anak kiri dan anak kanan [1]. Struktur ini banyak digunakan karena efisiensinya dalam operasi pencarian, penyisipan, dan penghapusan data. Dalam praktiknya, sebuah binary tree perlu ditelusuri (traversal) agar seluruh datanya dapat diakses secara berurutan. Terdapat tiga metode utama penelusuran binary tree, yaitu Preorder (akar–kiri–kanan), Inorder (kiri–akar–kanan), dan Postorder (kiri–kanan–akar). Masing-masing metode memiliki kegunaan yang berbeda tergantung pada kebutuhan pemrosesan datanya.

Selain tree, Graph juga merupakan struktur data yang tidak kalah penting. Graph adalah jenis struktur data umum yang susunan datanya tidak berdekatan satu sama lain (non-linier). Graph terdiri dari kumpulan simpul berhingga untuk menyimpan data dan antara dua buah simpul terdapat hubungan saling keterkaitan [2]. Untuk mengakses seluruh simpul pada sebuah graph, diperlukan algoritma penelusuran, di antaranya Depth First Search (DFS) yang menelusuri graf secara mendalam terlebih dahulu menggunakan pendekatan rekursif, dan Breadth First Search (BFS) yang menelusuri graf secara melebar menggunakan bantuan queue.

1.2 Tujuan Praktikum

1. Memahami konsep struktur data non-linier, khususnya Binary Tree dan Graph, serta perbedaannya dengan struktur data linier.
2. Mengimplementasikan operasi dasar pada Binary Tree, seperti pembuatan node, penambahan simpul, penghitungan jumlah node, serta penelusuran (traversal) dengan metode Preorder, Inorder, dan Postorder.
3. Memahami dan menerapkan algoritma penelusuran graf (Graph Traversal) menggunakan metode Depth First Search (DFS) dan Breadth First Search (BFS).
4. Mengembangkan kemampuan pemrograman dalam merepresentasikan struktur data tree dan graph ke dalam bahasa Java secara terstruktur.

1.3 Manfaat Praktikum

1. Memberikan pemahaman praktis mengenai cara kerja Binary Tree dan Graph, sehingga mahasiswa mampu memilih struktur data yang tepat sesuai dengan permasalahan yang dihadapi.
2. Melatih logika pemrograman dan kemampuan problem solving dalam mengimplementasikan algoritma penelusuran (traversal) pada struktur data non-linier.
3. Menjadi dasar yang kuat untuk mempelajari materi lanjutan seperti Binary Search Tree, Heap, Shortest Path, serta penerapan struktur data pada bidang seperti database, jaringan komputer, dan kecerdasan buatan.

BAB II

PEMBAHASAN

2.1 Node_2511532015

Kelas ini merupakan fondasi utama dalam pembuatan struktur data Binary Tree (Pohon Biner) pada program ini. Secara sederhana, kelas ini berfungsi sebagai cetakan (blueprint) untuk membuat sebuah simpul (node). Dalam konsep binary tree, setiap simpul hanya boleh memiliki maksimal dua cabang anak, yaitu anak di sebelah kiri dan anak di sebelah kanan. Oleh karena itu, kelas ini tidak hanya menyimpan nilai data, tetapi juga menyimpan "alamat" atau referensi ke mana cabang kiri dan kanannya mengarah.

```
package pekan9_2511532015;

public class Node_2511532015 {
    int data_2015;
    Node_2511532015 left_2015;
    Node_2511532015 right_2015;
    public Node_2511532015 (int data_2015) {
        this.data_2015 = data_2015;
        left_2015 = null;
        right_2015 = null;
    }
    public void setLeft_2015 (Node_2511532015 node) {
        if (right_2015 == null)
            left_2015 = node;
    }
    public void setRight_2015 (Node_2511532015 node) {
        if (right_2015 == null)
            right_2015 = node;
    }
    public Node_2511532015 getLeft_2015() {
        return left_2015;
    }
    public Node_2511532015 getRight_2015() {
        return right_2015;
    }
    public int getData_2015() {
        return data_2015;
    }
    public void setData_2015 (int data_2015) {
        this.data_2015 = data_2015;
    }
    void printPreorder (Node_2511532015 node) {
        if (node == null)
            return;
        System.out.print(node.data_2015 + " ");
    }
}
```

```

        printPreorder (node.left_2015);
        printPreorder (node.right_2015);
    }
    void printPostorder (Node_2511532015 node) {
        if (node == null)
            return;
        printPostorder (node.left_2015);
        printPostorder (node.right_2015);
        System.out.print(node.data_2015 + " ");
    }
    void printInorder (Node_2511532015 node) {
        if (node == null)
            return;
        printInorder (node.left_2015);
        System.out.print(node.data_2015 + " ");
        printInorder (node.right_2015);
    }
    public String print () {
        return this.print ("",true,"");
    }
    public String print (String prefix, boolean isTail,String sb)
    {
        if (right_2015 != null) {
            right_2015.print (prefix + (isTail ? "| " : " "),
            false, sb);
        }
        System.out.println(prefix+(isTail? "\\--":"/--
")+data_2015);
        if (left_2015!=null) {
            left_2015.print (prefix + (isTail ? " ": "| "),
            true, sb);
        }
        return sb;
    }
}

```

2.1.1 Atribut dan Konstruktor:

- Data_2015: bertipe integer yang berfungsi untuk menyimpan nilai atau informasi pada simpul tersebut.
- Left_2015 dan right_2015: bertipe kelas Node_2511532015 yang berfungsi sebagai penunjuk (pointer) yang menghubungkan simpul saat ini dengan simpul angka di sebelah kiri dan kanannya.

2.1.2 Ketika sebuah simpul dibuat dan diberikan sebuah nilai data, konstruktor akan secara otomatis mengatur agar left_2015 dan

right_2015 bernilai null yang menandakan bahwa simpul yang baru dibuat tersebut belum memiliki cabang anak sama sekali.

2.1.3 Kelas ini menyediakan method setter untuk menghubungkan simpul saat ini dengan simpul lainnya. Sedangkan simpul getter berfungsi untuk mengambil dan membaca informasi dari simpul tersebut, baik itu mengambil nilai maupun mengambil referensi ke simpul left dan right. Method setData_2015 juga ada apabila ingin mengubah nilai data pada simpul yang sudah terlanjur dibuat.

2.1.4 Method penelusuran pohon (Tree Traversal)

- printPreorder: method ini menelusuri pohon dengan urutan Akar-Left-Right. Artinya data pada simpul induk akan dicetak terlebih dahulu baru kemudian dilanjutkan dengan cabang kiri dan cabang kanannya.
- printPreorder: urutannya Left-Akar-Right. Method ini akan mencetak data secara berurutan dari nilai terkecil ke terbesar.
- printPostorder: urutannya Left-Right-Akar. Pada method ini, simpul induk baru dicetak setelah seluruh cabang selesai dicetak.

2.1.5 Method print(): method ini mencetak data secara berurutan seperti traversal, berfungsi untuk menggambarkan struktur pohon secara visual di layar. Method ini menggunakan parameter tambahan seperti prefix (untuk membuat garis penghubung) dan isTail (untuk menentukan apakah simpul tersebut adalah cabang paling bawah atau bukan). Hasilnya, struktur binary tree akan terlihat seperti pohon yang rebah ke samping.

2.2 BTree_2511532015

Kelas ini berfungsi sebagai pengendali utama dari struktur Binary Tree. Kelas ini bertanggung jawab untuk mengelola Kumpulan simpul (node), mengatur hubungan antar simpul, serta menyediakan berbagai method operasional yang dapat digunakan untuk memanipulasi pohon tersebut.

```
package pekan9_2511532015;  
  
public class BTree_2511532015 {  
    private Node_2511532015 root_2015;  
    private Node_2511532015 currentNode_2015;
```

```

public BTree_2511532015() {
    root_2015 = null;
}
public boolean search_2015 (int data_2015) {
    return search_2015 (root_2015, data_2015);
}
private boolean search_2015 (Node_2511532015 node, int
data_2015) {
    if (node.getData_2015()== data_2015)
        return true;
    if (node.getLeft_2015() != null)
        if (search_2015 (node.getLeft_2015(), data_2015))
            return true;
    if (node.getRight_2015() != null)
        if (search_2015 (node.getRight_2015(), data_2015))
            return true;
    return false;
}
public void printInOrder() {
    root_2015.printInorder(root_2015);
}
public void printPreOrder() {
    root_2015.printPreorder(root_2015);
}
public void printPostOrder() {
    root_2015.printPostorder(root_2015);
}
public Node_2511532015 getRoot_2015() {
    return root_2015;
}
public boolean isEmpty() {
    return root_2015 == null;
}
public int countNodes () {
    return countNodes (root_2015);
}
private int countNodes (Node_2511532015 node) {
    int count = 1;
    if (node == null) {
        return 0;
    } else {
        count += countNodes (node.getLeft_2015());
        count += countNodes (node.getRight_2015());
        return count;
    }
}
public void print_2015() {
    root_2015.print();
}
public Node_2511532015 getCurrent_2015() {
    return currentNode_2015;
}
public void setCurrent_2015 (Node_2511532015 node) {
    this.currentNode_2015 = node;
}
public void setRoot_2015(Node_2511532015 root_2015) {
    this.root_2015 = root_2015;
}

```

```
}  
}
```

2.2.1 Atribut dan Konstruktor

- `Root_2015`: merepresentasikan simpul paling atas atau akar (root) dari pohon. Semua penelusuran dan operasi pada pohon akan selalu dimulai dari simpul ini. Diinisialisasikan dengan nilai null untuk memastikan bahwa setiap kali objek pohon baru dibuat, pohon tersebut dalam keadaan kosong.
- `correntNode_2015`: berfungsi sebagai penunjuk (pointer) untuk menandai simpul tertentu yang sedang menjadi fokus atau posisi aktif saat ini.

2.2.2 `Method search_2015`: untuk mencari apakah suatu nilai data tertentu ada di dalam pohon dengan menggunakan teknik memanggil dirinya sendiri. Logika dimulai dari akar: jika data pada simpul saat ini cocok dengan data yang dicari, maka method akan mengembalikan nilai true. Dan jika tidak, method akan secara otomatis melanjutkan pencarian ke cabang sebelah left dan jika belum ditemukan maka pindah ke cabang right. Jika seluruh cabang telah diperiksa dan tidak ditemukan maka akan mengembalikan nilai false.

2.2.3 `Method Traversal Wrappers (Penelusuran)`: terdapat tiga method public (`printInorder()`, `printPreorder()`, dan `printPostorder()`) yang dimana method ini bertindak sebagai wrapper yang memanggil method penelusuran yang sudah didefinisikan di dalam kelas `Node`, dengan memberikan root sebagai titik awal penelusuran. Dengan adanya method ini, penggunaan `BTree` tidak perlu repot mengakses node secara manual untuk melakukan penelusuran.

- `isEmpty()`: untuk memeriksa apakah tree sedang kosong atau tidak, dengan mengecek apakah root bernilai null.
- `countNodes()`: digunakan untuk menghitung total jumlah simpul yang ada di dalam pohon. Logikanya: jumlah total node sama dengan 1 (untuk node saat ini) ditambah jumlah node di cabang kiri, ditambah dengan cabang node di sebelah kanan. Jika

method menemukan node yang bernilai null maka akan mengembalikan nilai 0 sebagai kondisi dasar agar rekursi berhenti.

- Print_2015: memanggil fungsi visualisasi dari kelas Node untuk mencetak bentuk pohon secara grafis.

2.3 BTreeDriver_2511532015

Kelas BTreeDriver_2511532015 merupakan kelas utama (main class) yang berfungsi sebagai pengendali eksekusi program. Kelas ini tidak mendefinisikan struktur data baru, melainkan bertugas untuk menguji dan mendemonstrasikan seluruh fungsi yang telah dibuat pada kelas Node_2511532015 dan BTree_2511532015. Melalui kelas inilah struktur Binary Tree dibentuk, simpul-simpul ditambahkan, dan berbagai operasi penelusuran dijalankan.

```
package pekan9_2511532015;

public class BTreeDriver_2511532015 {
    public static void main(String[] args) {
        // membuat pohon
        BTree_2511532015 tree_2015 = new BTree_2511532015();
        System.out.print("Jumlah Simpul awal pohon: ");
        System.out.println(tree_2015.countNodes());
        // menambahkan simpul data 1
        Node_2511532015 root_2015 = new Node_2511532015(1);
        // menjadikan simpul 1 sebagai root
        tree_2015.setRoot_2015(root_2015);
        System.out.println("Jumlah simpul jika hanya ada root");
        System.out.println(tree_2015.countNodes());
        Node_2511532015 node2_2015 = new Node_2511532015(2);
        Node_2511532015 node3_2015 = new Node_2511532015(3);
        Node_2511532015 node4_2015 = new Node_2511532015(4);
        Node_2511532015 node5_2015 = new Node_2511532015(5);
        Node_2511532015 node6_2015 = new Node_2511532015(6);
        Node_2511532015 node7_2015 = new Node_2511532015(7);
        Node_2511532015 node8_2015 = new Node_2511532015(8);
        Node_2511532015 node9_2015 = new Node_2511532015(9);
        root_2015.setLeft_2015(node2_2015);
        node2_2015.setLeft_2015 (node4_2015);
        node2_2015.setRight_2015 (node5_2015);
        node4_2015.setRight_2015 (node8_2015);
        root_2015.setRight_2015(node3_2015);
        node3_2015.setLeft_2015 (node6_2015);
        node3_2015.setRight_2015 (node7_2015);
        node6_2015.setLeft_2015 (node9_2015);
        // set root_2015
```

```

tree_2015.setCurrent_2015(tree_2015.getRoot_2015());
System.out.println("menampilkan simpul terakhir:");
System.out.println(tree_2015.getCurrent_2015().getData_2015()
);
System.out.println("Jumlah simpul; setelah simpul 7
ditambahkan");
System.out.println(tree_2015.countNodes());
System.out.println("InOrder: ");
tree_2015.printInOrder();
System.out.println("\nPreorder: ");
tree_2015.printPreOrder();
System.out.println("\nPostorder: ");
tree_2015.printPostOrder();
System.out.println("\nMenampilkan simpul dalam bentuk
pohon");
tree_2015.print_2015();
}
}

```

2.3.1 Program dimulai dengan membuat sebuah objek BTree_2511532015 yang diberi nama tree_2015. Pada tahap ini, pohon masih dalam keadaan kosong. Untuk membuktikan hal tersebut, program langsung memanggil metode countNodes() yang akan menghasilkan nilai 0. Selanjutnya, sebuah simpul baru dengan nilai data 1 dibuat dan dijadikan sebagai simpul akar (root) menggunakan metode setRoot_2015(). Setelah akar ditetapkan, jumlah simpul berubah menjadi 1, yang menandakan bahwa pohon kini memiliki satu simpul utama.

2.3.2 Setelah akar terbentuk, program membuat delapan simpul tambahan dengan nilai data dari 2 hingga 9. Simpul-simpul ini kemudian dihubungkan satu sama lain untuk membentuk struktur pohon yang spesifik menggunakan metode setLeft_2015() dan setRight_2015(). Berikut adalah susunan hierarki yang dibentuk:

- Simpul 1 (akar) memiliki anak kiri 2 dan anak kanan 3.
- Simpul 2 memiliki anak kiri 4 dan anak kanan 5.
- Simpul 4 memiliki anak kanan 8 (anak kirinya kosong).
- Simpul 3 memiliki anak kiri 6 dan anak kanan 7.
- Simpul 6 memiliki anak kiri 9 (anak kanannya kosong).

Dengan struktur ini, terbentuklah sebuah binary tree yang tidak seimbang (unbalanced tree) dengan kedalaman yang berbeda-beda pada setiap cabangnya.

2.3.3 Setelah seluruh struktur pohon selesai dibentuk, program mengatur simpul aktif (current node) ke simpul akar menggunakan metode `setCurrent_2015()`, lalu menampilkan nilai data dari simpul aktif tersebut, yaitu 1. Selanjutnya, metode `countNodes()` dipanggil kembali untuk menghitung total simpul yang telah ditambahkan, dan hasilnya adalah 9 simpul.

2.3.4 Bagian inti dari program ini adalah demonstrasi tiga jenis penelusuran pohon. Program memanggil metode `printInOrder()`, `printPreOrder()`, dan `printPostOrder()` secara berurutan. Masing-masing metode akan menghasilkan urutan pencetakan yang berbeda tergantung pada pola kunjungan yang digunakan, yaitu:

- `InOrder` akan mencetak dengan pola kiri-akar-kanan.
- `PreOrder` akan mencetak dengan pola akar-kiri-kanan.
- `PostOrder` akan mencetak dengan pola kiri-kanan-akar

2.3.5 Sebagai penutup, program memanggil metode `print_2015()` yang akan menampilkan struktur pohon dalam bentuk visual grafis di layar konsol. Visualisasi ini sangat membantu untuk memverifikasi apakah hubungan antar simpul yang telah dibuat sudah sesuai dengan yang diharapkan, karena kita dapat melihat secara langsung posisi setiap simpul, mana yang menjadi akar, cabang kiri, cabang kanan, serta simpul daun (leaf).

2.3.6 Outputnya:

```
Console X
<terminated> BTreeDriver_2511532015 (1) [Java Application] C:\Users\USER\p2\pool\plugins\or
Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
menampilkan simpul terakhir:
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
Preorder:
1 2 4 8 5 3 6 9 7
Postorder:
8 4 5 2 9 6 7 3 1
Menampilkan simpul dalam bentuk pohon
| /--7
| /--3
| | \--6
| | \--9
\--1
| /--5
\--2
| /--8
\--4
```

Gambar 2.3.1: Hasil Output kode program BTree_2511532015

Pada awal program, pohon berada dalam keadaan kosong dengan jumlah simpul 0, namun setelah simpul akar (root) dengan nilai 1 ditambahkan, jumlah simpul berubah menjadi 1 dan simpul aktif yang ditampilkan juga bernilai 1. Setelah seluruh simpul dari 2 hingga 9 ditambahkan ke dalam struktur pohon, metode penghitungan simpul berhasil mendeteksi total 9 simpul yang terhubung. Selanjutnya, program melakukan tiga jenis penelusuran (traversal) yang menghasilkan urutan berbeda sesuai dengan aturannya; metode InOrder mencetak urutan 4 8 2 5 1 9 6 3 7 dengan pola kiri-akar-kanan, PreOrder mencetak 1 2 4 8 5 3 6 9 7 dengan pola akar-kiri-kanan, dan PostOrder mencetak 8 4 5 2 9 6 7 3 1 dengan pola kiri-kanan-akar di mana nilai akar dicetak paling terakhir. Terakhir, program memvisualisasikan struktur pohon dalam bentuk teks grafis yang secara jelas menggambarkan hierarki data, di mana simpul 1 berada di puncak sebagai akar, dengan cabang kiri yang terdiri dari simpul 2, 4, 8, dan 5, serta cabang kanan yang terdiri dari simpul 3, 6, 9, dan 7.

2.4 GraphTraversal_2511532015

Kelas GraphTraversal_2511532015 merupakan implementasi dari struktur data Graph (graf) beserta algoritma penelusurannya. Jika pada

Binary Tree kita mengenal hubungan induk dan anak yang bersifat hierarkis, maka pada Graph, setiap simpul (vertex/node) dapat terhubung dengan simpul lainnya secara bebas tanpa mengenal tingkatan. Kelas ini dirancang untuk merepresentasikan graf tak berarah (undirected graph), di mana hubungan antara dua simpul bersifat dua arah, serta menyediakan dua algoritma penelusuran utama, yaitu Depth First Search (DFS) dan Breadth First Search (BFS).

```
package pekan9_2511532015;
import java.util.*;
public class GraphTraversal_2511532015{
    private Map<String, List<String>> graph_2015 = new
    HashMap<>();

    //menambahkan edge (graf tak berarah)
    public void addEdge_2015 (String node1, String node2) {
        graph_2015.putIfAbsent (node1, new ArrayList<>());
        graph_2015.putIfAbsent (node2, new ArrayList<>());
        graph_2015.get(node1).add(node2);
        graph_2015.get(node2).add(node1);
    }
    //menampilkan graf awal
    public void printGraph() {
        System.out.println("Graf awal (Adjacency List): ");
        for (String node : graph_2015.keySet()) {
            System.out.print(node + "->");
            List<String> neighbors = graph_2015.get(node);
            System.out.println(String.join(" ", neighbors));
        }
        System.out.println();
    }
    //DFS rekursif
    public void dfs_2015 (String start) {
        Set<String> visited = new HashSet<>();
        System.out.println("Penelusuran DFS: ");
        dfsHelper(start, visited);
        System.out.println();
    }
    private void dfsHelper (String current, Set<String>
    Visited) {
        if (Visited.contains (current)) return;
        Visited.add(current);
        System.out.print(current + " ");
        for (String neighbor: graph_2015.getDefault(current,
        new ArrayList<>())) {
            dfsHelper(neighbor, Visited);
        }
    }
    //BFS iteratif
    public void bfs_2015 (String start) {
        Set<String> visited = new HashSet<>();
        Queue<String> queue = new LinkedList<>();
```

```

        queue.add(start);
        visited.add(start);
        System.out.println("Penelusuran BFS: ");
        while (!queue.isEmpty()) {
            String current = queue.poll();
            System.out.print(current+ " ");
            for (String neighbor : graph_2015.getDefault
(current, new ArrayList<>())) {
                if (!visited.contains(neighbor)) {
                    queue.add(neighbor);
                    visited.add(neighbor);
                }
            }
        }
        System.out.println();
    }
    //main
    public static void main(String[] args) {
        GraphTraversal_2511532015 graph_2015 = new
GraphTraversal_2511532015();

        //contoh graf : A-B, A-C, B-D, B-E
        graph_2015.addEdge_2015("A", "B");
        graph_2015.addEdge_2015("A", "C");
        graph_2015.addEdge_2015("B", "D");
        graph_2015.addEdge_2015("B", "E");
        //cetak graf awal
        System.out.println("Graf Awal adalah: ");
        graph_2015.printGraph();
        //lakukan penelusuran
        graph_2015.dfs_2015("A");
        graph_2015.bfs_2015("A");
    }
}

```

2.4.1 Representasi Graf: Untuk menyimpan struktur graf, kelas ini menggunakan atribut `graph_2015` yang bertipe `Map<String, List<String>>` dengan implementasi `HashMap`. Pendekatan ini dikenal dengan istilah Adjacency List (Daftar Ketetanggaan), di mana setiap simpul (direpresentasikan sebagai `String`) akan dipetakan ke dalam sebuah `List` yang berisi simpul-simpul tetangganya. Penggunaan `HashMap` dipilih karena efisiensinya dalam menyimpan dan mengambil data berdasarkan kunci (key) berupa nama simpul.

2.4.2 Method `addEdge_2015()`: berfungsi untuk menambahkan hubungan atau sisi (edge) antara dua buah simpul. Karena graf yang dibuat

adalah graf tak berarah, maka ketika simpul A dihubungkan dengan simpul B, simpul B juga harus dihubungkan kembali ke simpul A. Untuk itu, metode ini menggunakan `putIfAbsent()` untuk memastikan kedua simpul sudah terdaftar di dalam `HashMap` sebelum akhirnya saling menambahkan satu sama lain ke dalam daftar tetangga masing-masing menggunakan perintah `add()`.

2.4.3 Method `printGraf()`: digunakan untuk menampilkan struktur graf yang telah dibuat dalam bentuk Adjacency List. Metode ini akan melakukan perulangan terhadap seluruh kunci (key) di dalam `HashMap`, lalu mencetak nama setiap simpul diikuti dengan tanda panah "`->`" dan daftar simpul tetangganya. Hasilnya, pengguna dapat melihat dengan jelas simpul mana saja yang saling terhubung.

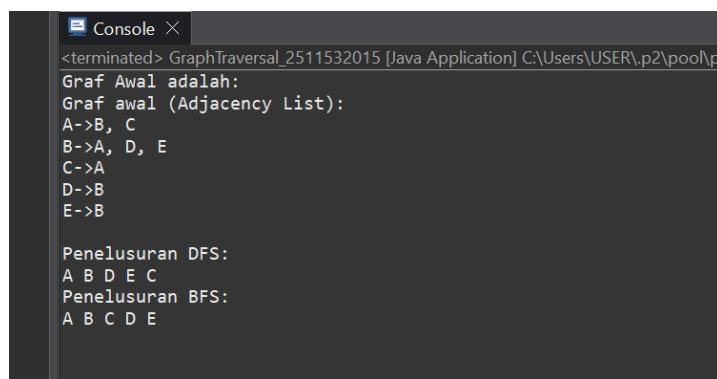
2.4.4 Method `dfs_2015()`: mengimplementasikan algoritma DFS yang menelusuri graf secara mendalam terlebih dahulu sebelum melebar. Algoritma ini bekerja secara rekursif dengan bantuan metode pembantu `dfsHelper()`. Untuk menghindari penelusuran berulang pada simpul yang sama (terutama pada graf yang memiliki siklus), metode ini menggunakan `HashSet` bernama `visited` untuk mencatat simpul-simpul yang sudah pernah dikunjungi. Logikanya, program akan mencetak simpul saat ini, menandainya sebagai sudah dikunjungi, lalu secara rekursif memanggil simpul tetangganya yang belum dikunjungi hingga seluruh cabang terdalam habis, baru kemudian kembali (backtracking) ke simpul sebelumnya.

2.4.5 Method `bfs_2015()`: mengimplementasikan algoritma BFS yang menelusuri graf secara melebar level demi level. Algoritma ini bekerja secara iteratif (menggunakan perulangan) dengan bantuan struktur data `Queue` (antrian) bertipe `LinkedList`. Prinsipnya mengikuti konsep FIFO (First In First Out): simpul awal dimasukkan ke dalam antrian, lalu selama antrian belum kosong, program akan mengambil simpul paling depan, mencetaknya, dan memasukkan seluruh tetangganya yang belum dikunjungi ke dalam

antrian. Sama seperti DFS, metode ini juga menggunakan HashSet bernama visited untuk mencegah penelusuran ganda.

2.4.6 Main method: program membuat sebuah objek graf dan menambahkan beberapa sisi untuk membentuk struktur sederhana, yaitu A terhubung dengan B dan C, serta B terhubung dengan D dan E. Setelah graf terbentuk, program pertama-tama menampilkan struktur Adjacency List, lalu melakukan penelusuran menggunakan metode DFS dimulai dari simpul A, dan diakhiri dengan penelusuran BFS juga dimulai dari simpul A. Perbandingan hasil dari kedua penelusuran ini nantinya akan menunjukkan perbedaan pola kunjungan antara algoritma yang mendalam (DFS) dengan algoritma yang melebar (BFS).

2.4.7 Outputnya:



```
<terminated> GraphTraversal_2511532015 [Java Application] C:\Users\USER\p2\pool\p
Graf Awal adalah:
Graf awal (Adjacency List):
A->B, C
B->A, D, E
C->A
D->B
E->B

Penelusuran DFS:
A B D E C
Penelusuran BFS:
A B C D E
```

Gambar 2.4.1: Hasil output kode program

GraphTraversal_2511532015

Berdasarkan hasil eksekusi program, output pertama menampilkan struktur graf dalam bentuk Adjacency List yang menunjukkan hubungan antar simpul. Simpul A terhubung dengan B dan C, simpul B terhubung dengan A, D, dan E, simpul C terhubung dengan A, simpul D terhubung dengan B, dan simpul E terhubung dengan B. Hubungan ini bersifat dua arah karena graf yang digunakan adalah graf tak berarah. Pada penelusuran DFS (Depth First Search), urutan kunjungan simpul adalah $A \rightarrow B \rightarrow D \rightarrow E \rightarrow C$, di mana algoritma menelusuri sedalam mungkin ke satu cabang (dari A ke B, lalu ke D, kemudian E) sebelum kembali dan mengunjungi cabang lainnya (C).

Sementara pada penelusuran BFS (Breadth First Search), urutan kunjungan adalah $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$, di mana algoritma mengunjungi semua simpul pada level yang sama terlebih dahulu (tetangga langsung A yaitu B dan C) sebelum berpindah ke level berikutnya (D dan E). Perbedaan hasil antara DFS dan BFS ini menunjukkan karakteristik masing-masing algoritma dalam menelusuri graf.

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilaksanakan, dapat disimpulkan bahwa:

1. Struktur data non-linier seperti Binary Tree dan Graph memiliki peran penting dalam merepresentasikan data yang memiliki hubungan hierarkis maupun jaringan, berbeda dengan struktur data linier seperti array atau linked list.
2. Pada Binary Tree, setiap simpul (node) maksimal hanya memiliki dua anak. Penelusuran (traversal) pada Binary Tree dapat dilakukan dengan tiga metode utama, yaitu Preorder (akar-kiri-kanan), Inorder (kiri-akar-kanan), dan Postorder (kiri-kanan-akar), yang masing-masing menghasilkan urutan data yang berbeda sesuai dengan kebutuhan pemrosesan.
3. Pada Graph, penelusuran simpul dapat dilakukan menggunakan algoritma Depth First Search (DFS) yang menelusuri graf secara mendalam hingga ke cabang paling ujung menggunakan pendekatan rekursif, dan Breadth First Search (BFS) yang menelusuri graf secara melebar level demi level menggunakan bantuan struktur data Queue (antrian).
4. Melalui praktikum ini, penulis berhasil mengimplementasikan konsep Binary Tree dan Graph Traversal ke dalam bahasa pemrograman Java. Hal ini memberikan pemahaman langsung mengenai cara kerja, perbedaan pola penelusuran, serta penerapan masing-masing algoritma dalam menangani struktur data non-linier.

DAFTAR PUSTAKA

- [1] R. Amalia, "Binary Tree adalah: Ciri-ciri, Fungsi, Jenis Struktur Data," IDwebhost, 20 November 2024. [Online]. Available: <https://idwebhost.com/blog/binary-tree-adalah/>. [Accessed 9 Juni 2026].
- [2] Trivusi, "Struktur Data Graph: Pengertian, Jenis, dan Kegunaannya," Trivusi, 16 September 2022. [Online]. Available: <https://www.trivusi.web.id/2022/07/struktur-data-graph.html>. [Accessed 9 Juni 2026].